



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по практической работе №4

по дисциплине

«Технология разработки программных приложений»

Тема: «Основы работы с Docker»

Выполнил:

Студент группы ИНБО-20-23

Боргачев Тимофей Максимович

Проверил:

ассистент Петрова А.А.

Москва 2025

ОГЛАВЛЕНИЕ

ПРАКТИЧЕСКАЯ РАБОТА №4. DOCKER	3
Введение	3
1. Образы.....	3
2. Изоляция	4
3. Работа с портами	5
4. Именованные контейнеры, остановка и удаление.....	8
5. Постоянное хранение данных.....	9
5.1 Тома.....	11
5.2 Монтирование директорий и файлов	12
6. Переменные окружения	13
7. Dockerfile.....	14
8. Индивидуальное задание	15
Вывод.....	17

ПРАКТИЧЕСКАЯ РАБОТА №4. DOCKER

Введение

В практической работе необходимо выполнить все шаги из разделов 1–7. В отчёт должны быть включены ответы на вопросы, выделенные курсивом, результаты выполнения команд из разделов 1–7, а также выполненное индивидуальное задание (раздел 8): листинг Dockerfile, а также команды сборки и запуска контейнера.

1. Образы

Для просмотра имеющихся образов можно выполнить команду *docker image*. Результат ввода команды представлен на рис. 1.

```
PS C:\Users\borga> docker images
REPOSITORY    TAG       IMAGE ID   CREATED   SIZE
PS C:\Users\borga>
```

Рисунок 1 – Отсутствие докер образов

Загрузка образа ubuntu с помощью команды *docker pull ubuntu* представлена на рис. 2.

```
PS C:\Users\borga> docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
2726e237d1a3: Pull complete
Digest: sha256:1e622c5f073b4f6bfad6632f2616c7f59ef256e96fe78bf6a595d1dc4376ac02
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

Рисунок 2 – Загрузка образа ubuntu

Теперь образ должен отобразиться. Повторный просмотр имеющихся образов представлен на рис. 3.

```
PS C:\Users\borga> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	602eb6fb314b	2 days ago	78.1MB

Рисунок 3 – Просмотр списка докер образов

Просмотр списка контейнеров с помощью команды *docker ps -a* представлен на рис. 4.

```
PS C:\Users\borga> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Рисунок 4 – Отсутствие докер контейнеров

2. Изоляция

При нескольких выполнениях команды *hostname* для просмотра информации о хостовой системе – результат не меняется. Подтверждение представлено на рис. 5.

```
PS C:\Users\borga> hostname
DESKTOP-BB86VVG
PS C:\Users\borga> hostname
DESKTOP-BB86VVG
```

Рисунок 5 – Выполнение hostname

При нескольких выполнениях команды *docker run ubuntu hostname* ответ отличается, потому что из одного образа были запущены два изолированных контейнера. Подтверждение представлено на рис. 6.

```
PS C:\Users\borga> docker run ubuntu hostname
581dacb78c92
PS C:\Users\borga> docker run ubuntu hostname
a05b32bf2453
```

Рисунок 6 – Просмотр информации hostname у контейнеров

Результат повторного запроса всех контейнеров представлен на рис. 7.

```
PS C:\Users\borga> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a05b32bf2453	ubuntu	"hostname"	4 minutes ago	Exited (0) 4 minutes ago		busy_stonebraker
581dacb78c92	ubuntu	"hostname"	4 minutes ago	Exited (0) 4 minutes ago		suspicious_haslett

Рисунок 7 – Повторный запрос контейнеров

Запуск `bash`'а из контейнера с помощью команды `docker run ubuntu bash` представлен на рис. 8.

```
PS C:\Users\borga> docker run ubuntu bash
PS C:\Users\borga>
```

Рисунок 8 – Запуск контейнера с завершением

Запуск `bash`'а из контейнера с помощью команды `docker run -it ubuntu bash` представлен на рис. 9.

```
PS C:\Users\borga> docker run -it ubuntu bash
root@4a1443364025:/#
```

Рисунок 9 – Запуск контейнера без завершения

3. Работа с портами

Загрузка образа `python` представлена на рис. 10.

```
PS C:\Users\borga> docker pull python
Using default tag: latest
latest: Pulling from library/python
23b7d26ef1d2: Pull complete
07d1b5af933d: Pull complete
1eb98adba0eb: Pull complete
b617a119f8a2: Pull complete
171e1bee1949: Pull complete
e25cca11fd29: Pull complete
739b86d2a778: Pull complete
Digest: sha256:9819e5616923079cc16af4a93d4be92c0c487c6e02fd9027220381f3e125d64a
Status: Downloaded newer image for python:latest
docker.io/library/python:latest
PS C:\Users\borga>
```

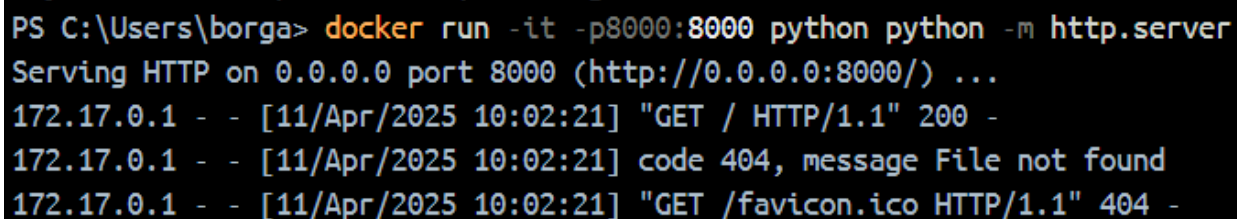
Рисунок 10 – Загрузка образа `python`

Запуск сервера без конфигурации портов представлен на рис. 11.

```
PS C:\Users\borga> docker run -it python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
^C
Keyboard interrupt received, exiting.
PS C:\Users\borga>
```

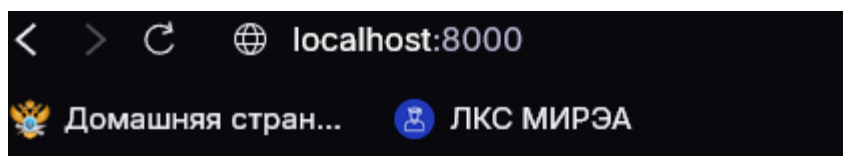
Рисунок 11 – Запуск сервера без конфигурации портов

Выполнение команды с пробросом *docker run -it -p8000:8000 python python -m http.server* портов представлено на рис. 12.



```
PS C:\Users\borga> docker run -it -p8000:8000 python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
172.17.0.1 - - [11/Apr/2025 10:02:21] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [11/Apr/2025 10:02:21] code 404, message File not found
172.17.0.1 - - [11/Apr/2025 10:02:21] "GET /favicon.ico HTTP/1.1" 404 -
```

Рисунок 12 – Запуск сервера с пробросом портов
Запущенный сервер представлен на рис. 13.



Directory listing for /

- [.dockerenv](#)
 - [bin@](#)
 - [boot/](#)
 - [dev/](#)
 - [etc/](#)
 - [home/](#)
 - [lib@](#)
 - [lib64@](#)
 - [media/](#)
 - [mnt/](#)
 - [opt/](#)
 - [proc/](#)
 - [root/](#)
 - [run/](#)
 - [sbin@](#)
 - [srv/](#)
 - [sys/](#)
 - [tmp/](#)
 - [usr/](#)
 - [var/](#)
-

Рисунок 13 – Запущенный сервер с пробросом портов
Запуск сервера на другом порте (8888) представлен на рис. 14.

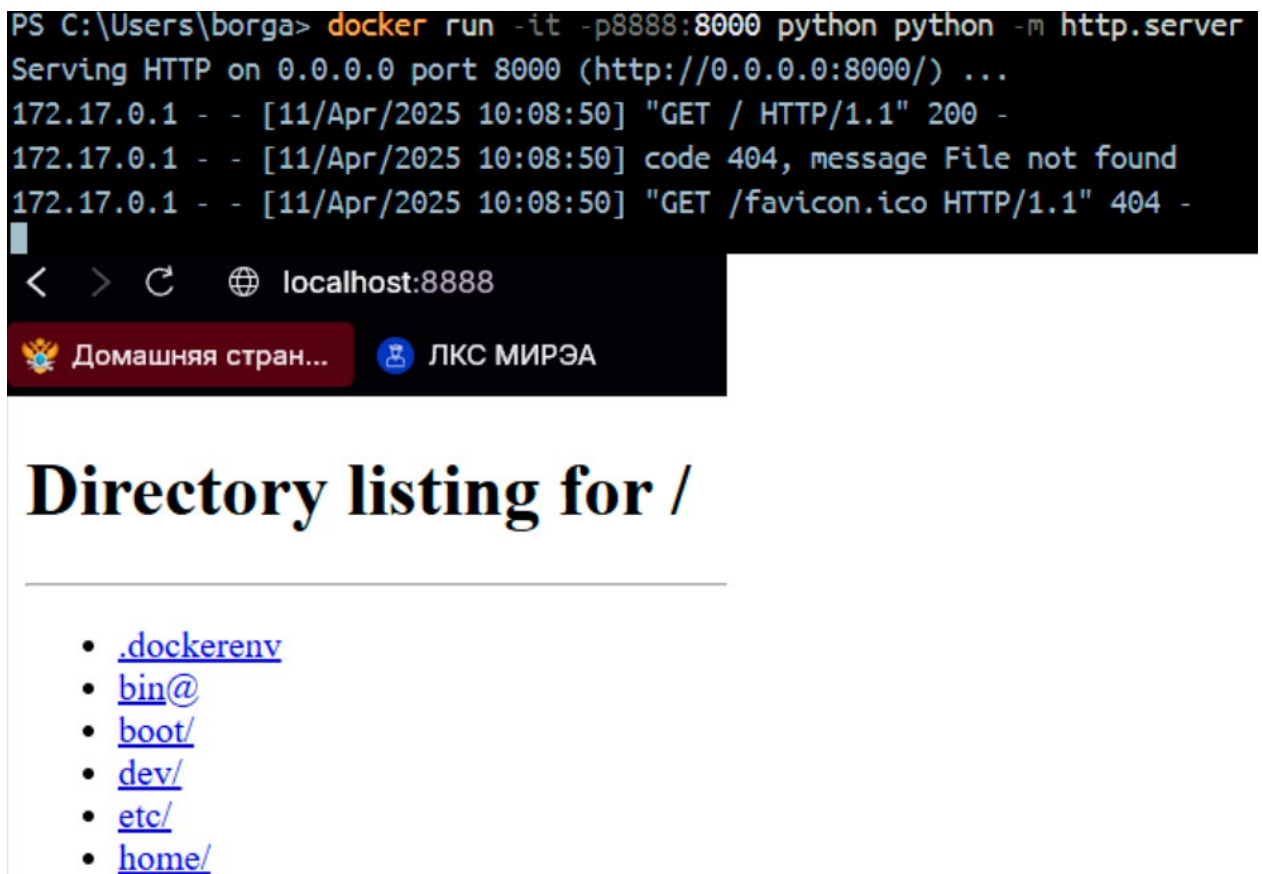


Рисунок 14 – Запуск сервера на порте 8888

4. Именованные контейнеры, остановка и удаление

Остановка контейнера представлена на рис. 15.

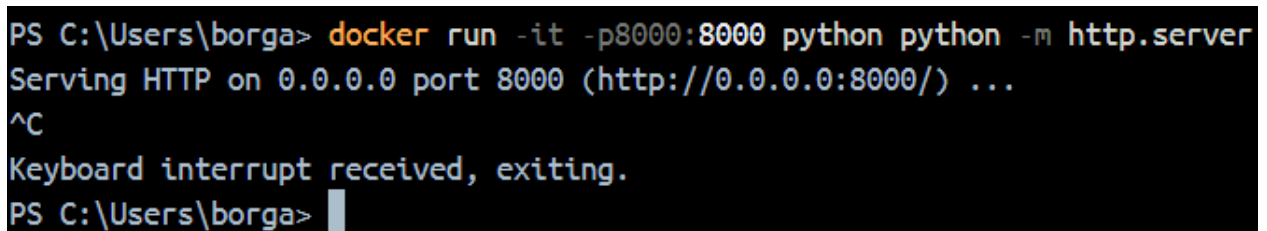


Рисунок 15 – Остановка контейнера

Запуск именованного контейнера в фоне представлен на рис. 16.

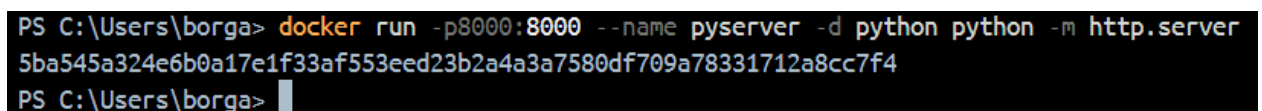


Рисунок 16 – Именованный контейнер запущенный в фоне

Проверим, что контейнер работает с помощью команды *docker ps | Select-String "pyserver"*. Результат представлен на рис. 17.


```
PS C:\Users\borga> docker ps | Select-String "pyserver"

5ba545a324e6   python   "python -m http.server"   2 minutes ago   Up 2 minutes   0.0.0.0:8000->8000/tcp   p
yserver
```

Рисунок 17 – Проверка работы контейнера в фоне

Просмотр логов контейнера представлен на рис. 18.

```
PS C:\Users\borga> docker logs pyserver
172.17.0.1 - - [11/Apr/2025 10:17:09] "GET / HTTP/1.1" 200 -
PS C:\Users\borga>
```

Рисунок 18 – Просмотр логов контейнера

Остановка запущенного в фоне контейнера представлена на рис. 19.

```
PS C:\Users\borga> docker stop pyserver
pyserver
PS C:\Users\borga>
```

Рисунок 19 – Остановка фонового контейнера

Попытка запуска существующего именованного контейнера представлена на рис. 20.

```
PS C:\Users\borga> docker run -it -p8000:8000 --name pyserver -d python python -m http.server
docker: Error response from daemon: Conflict. The container name "/pyserver" is already in use by container "5
ba545a324e6b0a17e1f33af553eed23b2a4a3a7580df709a78331712a8cc7f4". You have to remove (or rename) that containe
r to be able to reuse that name.

Run 'docker run --help' for more information
PS C:\Users\borga>
```

Рисунок 20 – Попытка запуска существующего именованного контейнера

Удаление именованного контейнера представлено на рис. 21.

```
PS C:\Users\borga> docker rm pyserver
pyserver
PS C:\Users\borga>
```

Рисунок 21 – Удаление именованного контейнера

5. Постоянное хранение данных

Выполним команду *docker run -p8000:8000 --name pyserver --rm -d python python -m http.server -d /mnt*. Флаг *-p8000:8000* – это проброс портов; *--name pyserver* – означает что контейнер - именованный с именем “pyserver”; -

-rm — означает, что контейнер удалится после остановки; *-m* — флаг указывающий на использование текущей директории; *-d* — флаг, указывающий на запуск контейнера в фоне. Команда, выполняемая в контейнере - */mnt*. Результат представлен на рис. 22.

```
PS C:\Users\borga> docker run -p8000:8000 --name pyserver --rm -d python python -m http.server -d /mnt
0cd5080ae8ccb60db235aafe4258ba87ced8d0911bba4d90698db04f34461d46
PS C:\Users\borga>
```

Рисунок 22 – Запуск контейнера с выполнением команды

Выполнение команды *docker exec -it pyserver bash* для того, чтобы попасть в уже запущенный контейнер представлено на рис. 23.

```
PS C:\Users\borga> docker exec -it pyserver bash
root@0cd5080ae8cc:/#
```

Рисунок 23 – Открытие bash в запущенном контейнере

Выполнение команды *cd mnt && echo "hello world" > hi.txt* представлено на рис. 24.

```
PS C:\Users\borga> docker exec -it pyserver bash
root@6d636ade49a2:/# cd mnt && echo "hello world" > hi.txt
root@6d636ade49a2:/mnt#
exit
PS C:\Users\borga>
```

Рисунок 24 – Выполнение bash команды в контейнере

Обновленная страница сервера представлена на рис. 25.

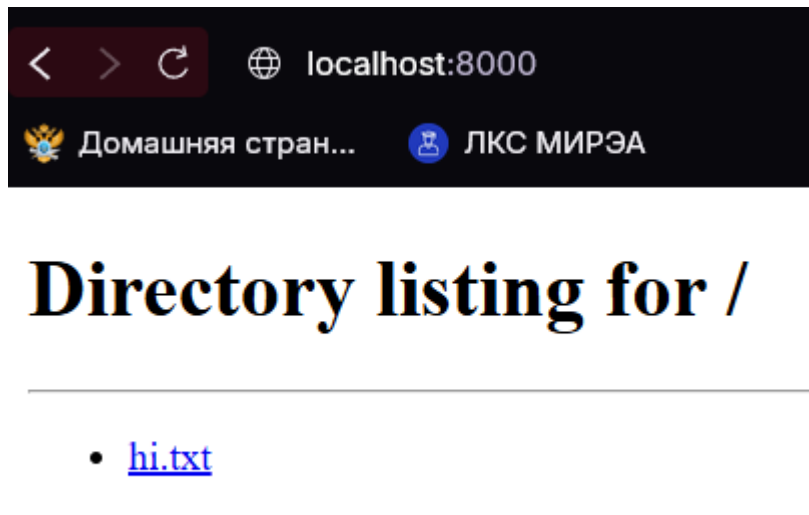


Рисунок 25 – Созданный файл hi.txt

Перезапустим контейнер и убедимся, что файл исчез. Выполнение команд и результат представлены на рис. 26.

```
PS C:\Users\borga> docker stop pyserver
pyserver
PS C:\Users\borga> docker run -p 8000:8000 --name pyserver --rm -d python python -m http.server -d /mnt
1720e0e2276289e682b9c50831c392cbf03fba2aec015bc4de01eebe1d5a507
PS C:\Users\borga>
```

Directory listing for /

Рисунок 26 – Перезапуск контейнера

5.1 Тома

Создадим контейнер с созданием тома с помощью команды `docker run -p 8000:8000 --rm --name pyserver -d -v myVolume:/mnt python python -m http.server -d /mnt`. Выполнение команды представлено на рис. 27.

```
borga@DESKTOP-BB86VVG MINGW64 ~
$ docker run -p 8000:8000 --rm --name pyserver -d -v myVolume:/mnt python python -m http.server -d /mnt
82fda884246c6c4d8afb55df434f91220dcf606d16581ef644c3fa7d083dedeb
borga@DESKTOP-BB86VVG MINGW64 ~
$ docker volume ls
DRIVER      VOLUME NAME
local       myVolume
```

Рисунок 27 – Запуск контейнера с созданием тома

Создание файла и проверка наличия тома представлена на рис. 28.

```
borga@DESKTOP-BB86VVG MINGW64 ~
$ docker exec -it pyserver bash
root@82fda884246c:/# cd mnt
root@82fda884246c:/mnt# ls
root@82fda884246c:/mnt# cd ..
root@82fda884246c:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@82fda884246c:/# cd mnt
root@82fda884246c:/mnt#
exit

borga@DESKTOP-BB86VVG MINGW64 ~
$ docker inspect -f "{{json .Mounts}}" pyserver
[{"Type": "volume", "Name": "myVolume", "Source": "/var/lib/docker/volumes/myVolume/_data", "Destination": "/mnt", "Driver": "local", "Mode": "z", "RW": true, "Propagation": ""}]
```

Рисунок 28 – Проверка томов, привязанных к контейнеру

5.2 Монтирование директорий и файлов

Остановим контейнер командой *docker stop pyserver*. Созданные директория и *host.txt* представлены на рис. 29.

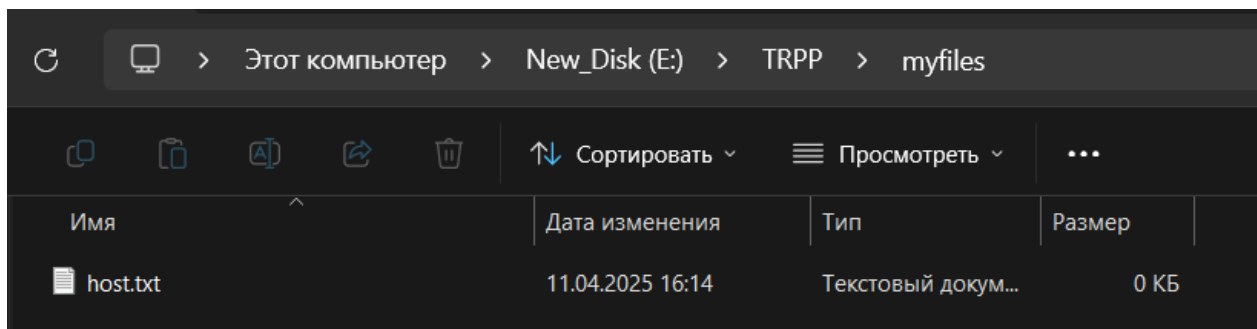


Рисунок 29 – Создание директории и файла

Запуск контейнера с монтированием созданной директории и вход в *bash* представлен на рис. 30.

```
PS C:\Users\borga> docker run -p 8000:8000 --rm --name pyserver -d -v "${currentDir}/myfiles:/mnt" python python -m http.server -d /mnt
6bbabc79970c4cc2b58f9bcfbcafb540e6fb53c475c103bc2c3bf8a50678016
PS C:\Users\borga> docker exec -it pyserver bash
```

Рисунок 30 – Запуск контейнера с монтированием myfiles

Вывод файлов в директории внутри докер контейнера с монтированными файлами представлен на рис. 31.

```
root@836f4557484a:/# cd mnt
root@836f4557484a:/mnt# ls
host.txt
root@836f4557484a:/mnt#
```

Рисунок 31 – Наличие файла в монтированной директории

Создание файла в монтированной директории и проверка его наличия

представлена на рис. 32.

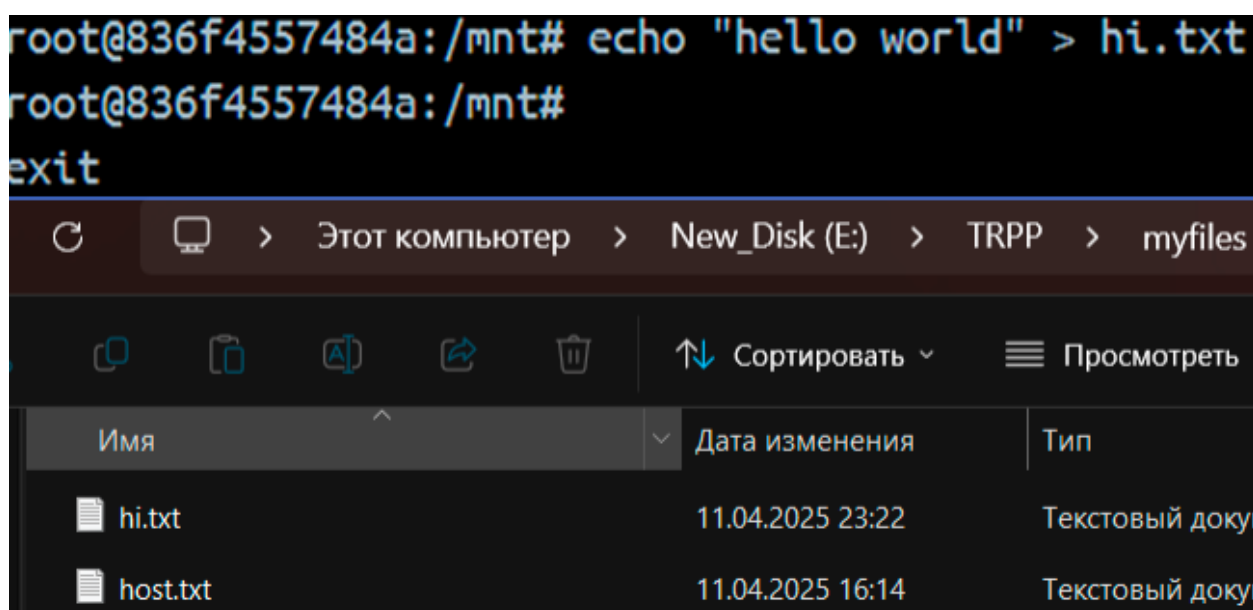


Рисунок 32 – Проверка созданного в монтированной директории файла
Монтирование файла с переименованием представлено на рис. 33.

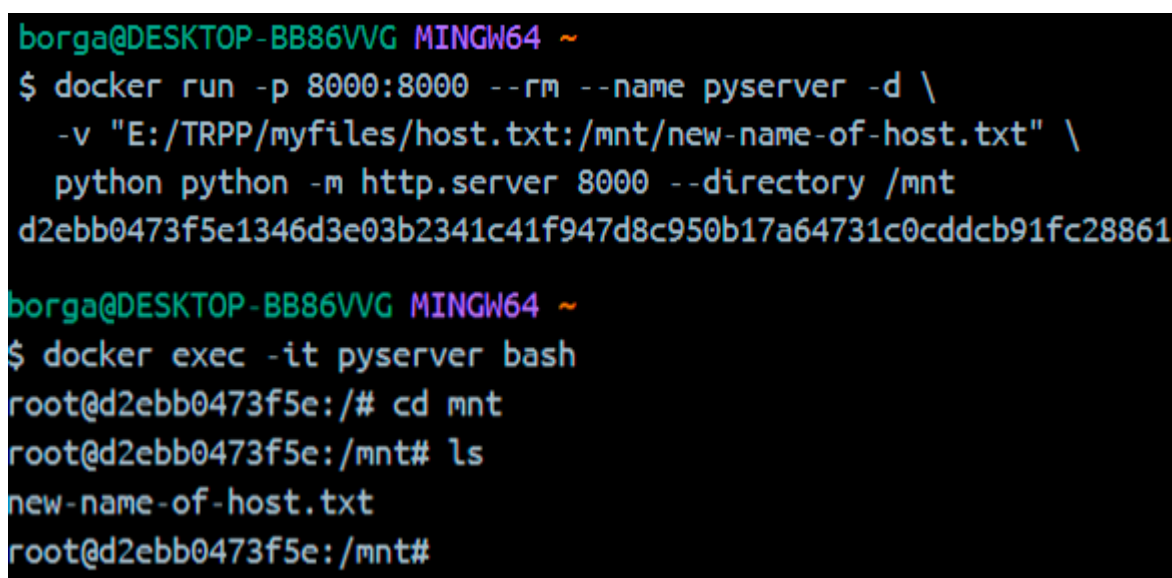


Рисунок 33 – Монтирование файла с переименованием

6. Переменные окружения

Создание и вывод переменных окружения представлен на рис. 34.

```
borga@DESKTOP-BB86VVG MINGW64 ~  
$ docker run -it --rm -e MIREA="ONE LOVE" ubuntu env  
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin  
HOSTNAME=b853723ee22b  
TERM=xterm  
MIREA=ONE LOVE  
HOME=/root
```

Рисунок 34 – Создание и вывод переменных окружения контейнера

7. Dockerfile

Созданный dockerfile представлен на рис. 35.

```
Dockerfile X  
E: > TRPP > myfiles > Dockerfile  
1 FROM ubuntu:20.04  
2 RUN apt update \  
3     && apt install -y python3 fortune \  
4     && cd /usr/bin \  
5     && ln -s python3 python  
6 RUN mkdir -p /mnt  
7 RUN /usr/games/fortune > /mnt/greeting-while-building.txt  
8 ADD ./data /mnt/data  
9 EXPOSE 80  
10 CMD ["python", "-m", "http.server", "80", "--directory", "/mnt"]
```

Рисунок 35 – Созданный Dockerfile

Процесс создания образа с помощью докер файла представлен на рис.

36.

```

barga@DESKTOP-BB86VVG MINGW64 ~
$ docker build -t mycoolimage "E:\TRPP\myfiles"

[+] Building 21.2s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 343B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:20.04    0.6s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafa15388214  4.4s
=> => resolve docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafa15388214  0.0s
=> => sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafa15388214 6.69kB / 6.69kB 0.0s
=> => sha256:c664f8f86ed5a386b0a340d981b8f81714e21a8b9c73f658c4bea56aa179d54a 424B / 424B 0.0s
=> => sha256:b7bab04fd9aa0c771e5720bf0cc7cbf993fd6946645983d9096126e5af45d713 2.30kB / 2.30kB 0.0s
=> => sha256:13b7e930469f6d3575a320709035c6acf6f5485a76abcf03d1b92a64c09c2476 27.51MB / 27.51MB 2.9s
=> => extracting sha256:13b7e930469f6d3575a320709035c6acf6f5485a76abcf03d1b92a64c09c2476 1.3s
=> [internal] load build context                                   0.0s
=> => transferring context: 26B                                       0.0s
=> [2/5] RUN apt update && apt install -y python3 fortune && cd /usr/bin && ln -s python3 python 14.7s
=> [3/5] RUN mkdir -p /mnt                                         0.3s
=> [4/5] RUN /usr/games/fortune > /mnt/greeting-while-building.txt 0.5s
=> [5/5] ADD ./data /mnt/data                                     0.1s
=> => exporting to image                                             0.4s
=> => exporting layers                                              0.4s
=> => writing image sha256:7a256da80ad8f87c966337b9c5ac6a0d49f7f69702322f06f4dcb72a17a65fd3 0.0s
=> => naming to docker.io/library/mycoolimage                      0.0s

View build details: docker_desktop://dashboard/build/desktop-linux/desktop-linux/dg15cfjf65ydt0h07l0gxpni

```

Рисунок 36 – Процесс создания образа

Запуск контейнера из созданного образа представлен на рис. 37.

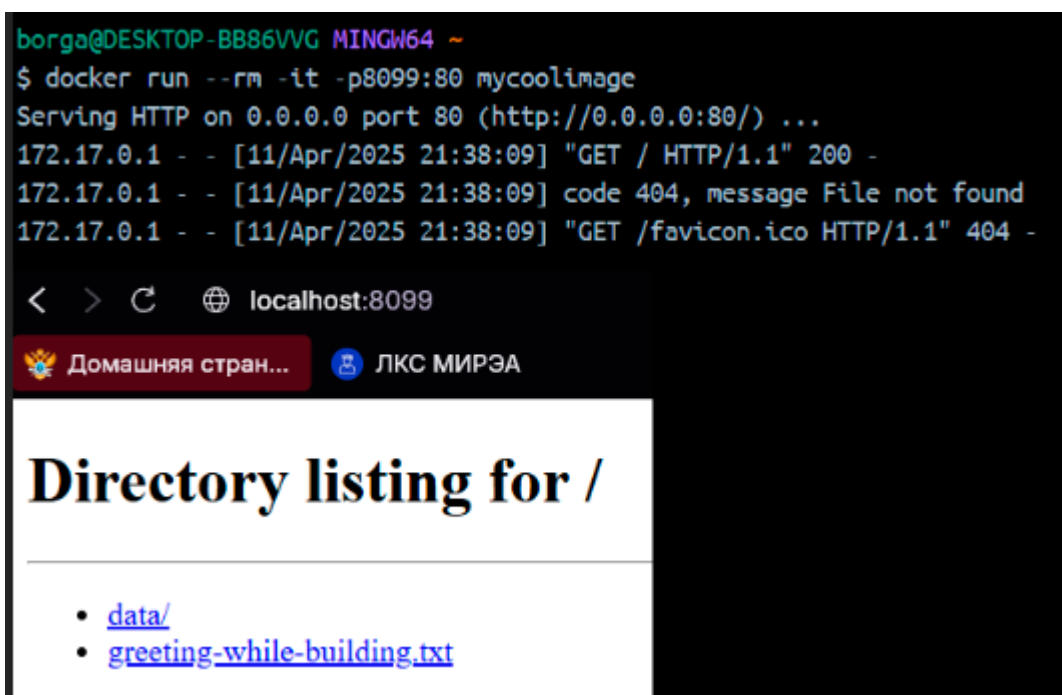


Рисунок 37 – Запуск контейнера из созданного образа

8. Индивидуальное задание

Для варианта 3 – измененный Dockerfile представлен на рис. 38.

```
Dockerfile X
E: > TRPP > myfile > Dockerfile
1 FROM ubuntu:20.04
2 RUN apt update && apt install -y zip python3
3 RUN mkdir -p /mnt/files
4 RUN cd /usr/bin && ln -s python3 python
5 EXPOSE 80
6 CMD ["python", "-m", "http.server", "80", "--directory", "/mnt/files"]
```

Рисунок 38 – Измененный докер файл в соответствии с вариантом 3

Создание файла student.txt в директории data представлено на рис. 39.

```
borga@DESKTOP-BB86VVG MINGW64 /e/TRPP
$ mkdir -p data

borga@DESKTOP-BB86VVG MINGW64 /e/TRPP
$ echo "ФИО: Боргачев Тимофей Максимович" > data/student.txt

borga@DESKTOP-BB86VVG MINGW64 /e/TRPP
$ echo "Группа: ИБ50-20-23" > data/student.txt

borga@DESKTOP-BB86VVG MINGW64 /e/TRPP
$ echo "Вариант: 3" > data/student.txt
```

Рисунок 39 – Создание файла student.txt

Процесс создания образа в соответствии с вариантом представлен на рис. 40.

```
borga@DESKTOP-BB86VVG MINGW64 /e/TRPP
$ docker build -t my-ubuntu-server .
[+] Building 22.9s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 251B
=> [internal] load metadata for docker.io/library/ubuntu:20.04
=> [auth] library/ubuntu:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/4] FROM docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafe15388214
=> [2/4] RUN apt update && apt install -y zip python3
=> [3/4] RUN mkdir -p /mnt/files
=> [4/4] RUN cd /usr/bin && ln -s python3 python
=> exporting to image
=> => exporting layers
=> => writing image sha256:8d50a7c2539962bc1ee008adf6a1297a31690c925c5bf78b3559e223ccb61400
=> => naming to docker.io/library/my-ubuntu-server
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/kqij2fzb4tfq7n1jbacgrftrc
```

Рисунок 40 – Создание образа по варианту

Результат запуска контейнера в соответствии с вариантом представлен

на рис. 41.



Рисунок 41 – Запуск контейнера по варианту

Вывод

В ходе выполнения практической работы по теме "Основы работы с Docker" были изучены и применены на практике основные концепции и команды Docker. Было выполнено следующее:

1. Изучена система управления образами Docker: загружены и просмотрены существующие образы, создан собственный образ.
2. Проверена изоляция контейнеров: продемонстрировано различие между хост-системой и контейнерами, запущенными из одного образа.
3. На практике рассмотрена работа с портами: организован проброс портов между хостом и контейнером, запущен веб-сервер внутри контейнера с доступом к нему с хост-машины.
4. Получены навыки работы с именованными контейнерами: 学会了 создавать, останавливать, запускать и удалять именованные контейнеры, а также просматривать их логи.
5. Изучены механизмы постоянного хранения данных в Docker: созданы тома, смонтированы директории и файлы хост-системы в контейнеры, проверена персистентность данных после перезапуска контейнеров.
6. Рассмотрена работа с переменными окружения внутри контейнеров.
7. Создан и протестирован первый Dockerfile для автоматизации создания

образов.

8. Выполнено индивидуальное задание: написан специализированный Dockerfile согласно варианту, создан соответствующий образ и успешно запущен контейнер, демонстрирующий требуемую функциональность.

В результате выполнения работы получены практические навыки работы с Docker - современной системой контейнеризации приложений, что является важным шагом в освоении технологий разработки программного обеспечения. Приобретенные знания могут быть применены при разработке и деплое реальных программных проектов.